

**T.C.
KASTAMONU ÜNİVERSİTESİ
MÜHENDİSLİK VE MİMARLIK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ**



NESNEYE YÖNELİK PROGRAMLAMA

PROJE RAPORU

Proje Sahibi	İrem AKGÜN
Okul No	234410151
Danışman	Ahmet Nusret ÖZALP
Ders Kodu	BSM204
Tarih	30.05.2026

Hook Tespit Aracı

Hook Tespit Aracı: RAII, Bellek Yönetimi ve İmza Tabanlı Analiz Kullanan Nesneye Yönelik C++ Gösterimi

Özet—Bu çalışma, nesneye yönelik programlama (NYP) ilkeleri kullanılarak C++17'de geliştirilen satır içi fonksiyon kanca tespit aracının tasarım ve uygulamasını sunmaktadır. Sistem; hem güvenlik yazılımlarında hem de kötü amaçlı yazılımlarda yaygın biçimde kullanılan satır içi kancanın bayt düzeyinde süreç belleği üzerinde çalışan, bağımsız bir uygulama içerisinde simüle edilip tespit edilmesini sağlamaktadır. Dört sınıf tanıtılmaktadır: düşük seviyeli Windows API bellek okuma/yazma işlemlerini soyutlayan MemoryManager; sanal bellek koruma değişikliklerine RAII deyimini uygulayan MemoryProtectionGuard; hedef fonksiyon üzerine yapay bir 0xE9 (görelî JMP) kancası kuran HookSimulator ve fonksiyon önsözlerinde bilinen kanca imzalarını tarayan HookDetector. Proje hiçbir harici araca gerek duymamakta olup hem g++ (MinGW) hem de MSVC ile C++17 altında derlenebilmektedir.

Anahtar Kelimeler—satır içi kanca, kanca tespiti, nesneye yönelik programlama, RAII, bellek yönetimi, Windows API, C++17, siber güvenlik, tersine mühendislik, yazılım güvenliği

I. GİRİŞ

Fonksiyon kancalama, çalışan bir sürecin davranışını makine kodu düzeyinde yürütme akışını yönlendirerek değiştirmeye olanak tanıyan, sistem programlamada temel bir tekniktir. Meşru kullanımları API izleme, antivirüsün ön yükleme katmanı ve hata ayıklama aracılığını kapsamaktadır. Düşmanca kullanım senaryoları arasında kötü niyetli faaliyetleri gizlemek veya program durumunu değiştirmek için sistem çağrılarını kesen rootkit'ler, kimlik bilgisi hırsızları ve oyun hileleri sayılabilir [1].

Çeşitli kancalama yöntemleri arasında satırıçi kancalama — hedef fonksiyonun ilk baytlarının koşulsuz bir atlama komutuyla üzerine yazılması — basitliği ve etkinliği nedeniyle en yaygın kullanılan yöntemdir. x86 ve x86-64 mimarilerinde 0xE9 işlem kodu görelî bir yakın atlamayı kodlamakta; toplam beş bayt gerektirmektedir: biri işlem kodu için, dördü ise hedef adrese olan işaretli uzaklık içindir [2].

Bu çalışma şu katkıları sunmaktadır: (1) dört iyi kapsüllenmiş sınıf kullanılarak satırıçi kanca simülatörü ve tespitçisinin temiz bir C++ uygulamasını sunmaktadır; (2) Windows sanal bellek koruma işlemleri bağlamında RAIL'in otomatik kaynak yönetimine uygulanmasını göstermektedir; (3) NYP, sistem programlama veya giriş düzeyinde siber güvenlik derslerine uygun, bağımsız pedagojik bir örnek sağlamaktadır.

II. İLGİLİ ÇALIŞMALAR

A. Fonksiyon Kancalama Teknikleri

Fonksiyon kancalama teknikleri geniş ölçüde dört kategoride sınıflandırılabilir: satırıçi (splice), içeri aktarma adres tablosu (IAT), sanal fonksiyon tablosu (VTable) ve donanım kesme noktaları kancaları [3]. Satırıçi kancalar yüklenmiş bir modülün kod bölümünü değiştirerek en fazla müdahaleye yol açan türlerdir. IAT kancaları yürütme koduna dokunmadan içeri aktarma çözümleme katmanındaki çağrılar yönlendirmektedir. VTable kancaları C++ sanal yönlendirme mekanizmalarını hedef almaktadır. Donanım kancaları ise CPU hata ayıklama yazmaçlarına dayanmakta ve yazılım bellekte hiçbir iz bırakmamaktadır.

0xE9 görelî JMP ve 0xFF 0x25 mutlak dolaylı JMP, Microsoft Detours [4] ve MinHook [5] gibi üretim kanca çerçevelerinde en sık karşılaşılan iki imzadır. Her iki imza da bu çalışmada uygulanan tespitçi tarafından hedef alınmaktadır.

B. Kanca Tespit Yaklaşımları

Tespit stratejileri üç kategoriye ayrılmaktadır: bellekteki fonksiyon önsözlerinin statik imza taraması; beklenmedik kontrol akışı transferlerini izleyen davranışsal analiz ve canlı belleği güvenilir bir disk içi görüntü ile karşılaştıran bütünlük doğrulama [6]. Bu çalışmada benimsenen statik imza taraması, bilinen kanca kalıpları için en basit ve en verimli yaklaşım olmakla birlikte polimorfik veya şifrelenmiş kancalara karşı savunmasızdır [7].

C. Sistem Programlamada NYP

Nesneye yönelik tasarım kalıplarının düşük seviyeli sistem koduna uygulanması, işletim sistemi sürücüler [8] ve gömülü bellek [9] bağlamlarında araştırılmıştır. Stroustrup tarafından C++ kaynak yönetim modelinin bir parçası olarak ilk kez resmîleştirilen RAIL, sistem kodunda manuel temizleme kalıplarıyla karşılaştırıldığında kaynak sızıntı oranlarını anlamlı ölçüde azalttığı gösterilmiştir [10].

III. SİSTEM MİMARİSİ

Araç, Tek Sorumluluk İlkesi etrafında organize edilen dört sınıftan oluşmaktadır: her sınıf tam olarak bir kaygının sahibidir. HookSimulator ve HookDetector'ın ikisi de bir MemoryManager örneğini

birleştirirken HookSimulator, kanca uygulama ve kaldırma işlemleri sırasında yığıta MemoryProtectionGuard nesnelerini dahili olarak oluşturmaktadır.

TABLO I. Sınıf Sorumlulukları ve Uygulanan NYP İlkeleri

Sınıf	Sorumluluk	Uygulanan NYP İlkeleri
MemoryManager	ReadProcessMemory / WriteProcessMemory çağrılarını soyutlar	Tek Sorumluluk, Kapsülleme
MemoryProtectionGuard	VirtualProtect yaşam döngüsünü yönetir; eski bayrakları yıkımda geri yükler	RAII, Kapsülleme, Kopyasız sahiplik
HookSimulator	Hedef fonksiyon üzerine yapay bir 0xE9 JMP kancası kurar ve kaldırır	Kapsülleme, RAII (Guard üzerinden), Kopyasız sahiplik
HookDetector	Fonksiyon önsözlerini bilinen kanca bayt imzaları için tarar	Tek Sorumluluk, Kapsülleme

main() fonksiyonu beş adımlı bir gösterim senaryosunu düzenlemektedir: temiz tarama, kanca uygulama, kanca sonrası tarama, kanca kaldırma ve son temiz tarama.

IV. SINIF TASARIMI VE UYGULAMASI

A. MemoryManager

MemoryManager iki const niteliğinde genel yöntem sağlamaktadır: readBytes() ve writeBytes(). Her ikisi de araç kendi adres uzayında çalıştığı için süreç tanıtıcısı olarak GetCurrentProcess() iletilerek sırasıyla Windows ReadProcessMemory ve WriteProcessMemory API'lerini sarmalar. Hata koşulları std::cerr'e bildirilmekte ve çağrılara boş vektör veya false dönüş değeri aracılığıyla iletilmektedir.

B. MemoryProtectionGuard (RAII)

MemoryProtectionGuard tek bir VirtualProtect çağrı çiftini kapsüllemektedir. Yapıcı istenen koruma bayraklarını uygular ve önceki bayrakları m_oldProtect'te saklar. Yıkıcı, guard başarıyla etkinleştirilmişse bu bayrakları koşulsuz olarak geri yükler. Bu, koruma ve geri yükleme çağrıları arasındaki bir istisna veya erken dönüş nedeniyle yürütme belleğinin hiçbir zaman yazılabilir durumda bırakılmamasını garantilemektedir [11].

C. HookSimulator

HookSimulator hedef fonksiyon adresini, kanca kurulumundan önce yedeklenen orijinal beş baytı, bir MemoryManager örneğini ve bir boole durum işaretçisini (m_isHooked) depolamaktadır. apply() yöntemi dört adımda ilerlemektedir: (1) orijinal baytları yedekle; (2) PAGE_EXECUTE_READWRITE ile yazma iznini aç; (3) yapay bir uzaklık için {0xE9, 0xDE, 0xAD, 0xBE, 0xEF} kanca bayt vektörünü oluştur; (4) baytları yaz. Guard blok sonunda kapsam dışına çıkar ve otomatik olarak PAGE_EXECUTE_READ'i geri yükler. Yıkıcı, m_isHooked true ise restore() çağırılmaktadır.

D. HookDetector

HookDetector, içsel MemoryManager'ını kullanarak SCAN_SIZE (5) baytı belirtilen adresten okuyan ve birinci baytı iki bilinen kanca imzasına karşı kontrol eden tek bir genel yöntem olan scan()'i açığa çıkarır. Başlangıç 0xE9 baytı görelili yakın JMP'yi, 0xFF 0x25 dizisi ise mutlak dolaylı JMP'yi göstermektedir.

V. YÜRÜTME SENARYOSU

Beş adımlı main() senaryosu öz doğrulamalı olmak üzere tasarlanmıştır: her tarama hem ham baytları hem de insan tarafından okunabilir bir kararı yazdırmaktadır.

TABLO II. Yürütme Adımı Özeti

Adım	Eylem	Okunan Birinci Bayt	Beklenen Karar
1	Kanca öncesi temiz tarama	Orijinal önsöz baytı (örn. 0x55 PUSH RBP)	Kanca tespit edilmedi
2	Kanca uygula (0xE9 yazıldı)	0xE9	Kanca başarıyla uygulandı
3	Kanca sonrası tarama	0xE9	0xE9 Görelili JMP tespit edildi
4	Orijinal baytları geri yükle	0xE9 → orijinal	Kanca kaldırıldı
5	Son temiz tarama	Orijinal önsöz baytı	Kanca tespit edilmedi

5. Adımın ardından hedefFonksiyon() bir hata olmaksızın çalıştırılarak geri yükleme döngüsünün bayt düzeyinde kusursuz gerçekleştiğinin ampirik kanıtı sağlanmaktadır.

VI. TARTIŞMA

A. Doğruluk ve Güvenlik

RAII guard tasarımı, PAGE_EXECUTE_READWRITE'in mümkün olan en kısa süre için tutulmasını garantilemektedir. apply()'da guard kapsamı, writeBytes() dönüş yapmasının hemen ardından sona ermektedir. Bu, başka bir iş parçacığının artık yazılabilir kod bölümüne yazabileceği pencereyi en aza indirmektedir.

B. Derleme ve Taşınabilirlik

Araç, Win32 API'sinden ReadProcessMemory, WriteProcessMemory, VirtualProtect ve GetCurrentProcess kullanımı nedeniyle yalnızca Windows işletim sisteminde çalışabilmektedir. g++ (MinGW) altında derleme komutu: g++ -o hook_detector main.cpp -static -std=c++17. Bu komut, taşınabilir tek bir yürütme dosyası üretmek için C++ çalışma zamanını statik olarak bağlamaktadır.

C. Sınırlılıklar

Araçın kapsamını sınırlandıran birçok kısıtlılık bulunmaktadır. Kanca uzaklığı yapay olup herhangi bir gerçek hedefe yürütme akışını yönlendirmemektedir. Tespitçi yalnızca fonksiyon önsözünün ilk beş baytını taramaktadır. Araç, hedef fonksiyonun beş bayttan kısa olduğu durumla başa çıkamamaktadır. Bunların yanında Kontrol Akışı Koruma veya PatchGuard aktif olan sistemlerde sistem DLL'lerinin kod bölümlerinin değiştirilmesi çekirdek düzeyinde engellenecektir.

D. Eğitimsel Değer

Proje beş NYP kavramını entegre bir bağlamda sergilemektedir: kapsülleme, tek sorumluluk, RAIİ, kopyasız sahiplik semantiği ve birleştirme. Bu aracı uygulayan öğrenciler; işaretçi aritmetiği, bellek koruma semantiği ve komut düzeyinde kod analizi konularında uygulamalı deneyim kazanmaktadır.

VII. SONUÇ

Bu çalışmada, dört nesneye yönelik sınıf kullanılarak C++17'de uygulanan bir satırıcı kanca tespit aracı sunulmuştur. Araç, gerçekçi bir kanca kurulum ve kaldırma döngüsünü simüle etmekte, bilinen kanca imzalarını taramakta ve tüm Windows API kaynak yaşam döngüsünü RAIİ aracılığıyla yönetmektedir. Harici bağımlılık gerektirmeksizin derlenmekte olup NYP veya sistem programlama dersleri için pedagojik açıdan kapsamlı bir referans niteliği taşımaktadır.

TEŞEKKÜR

Yazarlar yapıcı geri bildirimlerinden dolayı hakeme teşekkür eder. Bu çalışma bir Nesneye Yönelik Programlama dersi proje ödevi kapsamında yürütülmüş olup herhangi bir dış fon desteği almamıştır.

KAYNAKLAR

- [1] J. Richter ve C. Nasarre, Windows via C/C++, 5. baskı. Redmond, WA: Microsoft Press, 2008.
- [2] Intel Corporation, Intel 64 ve IA-32 Mimarileri Yazılım Geliştirici El Kitabı, cilt 2A. Santa Clara, CA: Intel, 2023.
- [3] P. Ferrie, "Daha fazla sanal makine emülatörüne saldırılar," Symantec Teknik Raporu, 2007.
- [4] G. Hunt ve D. Brubacher, "Detours: Win32 fonksiyonlarının ikili şekilde engellenmesi," in 3. USENIX Windows NT Sempozyumu, Seattle, WA, 1999, ss. 135–143.
- [5] T. Ono, "MinHook: Windows için minimalist x86/x64 API kancalama kütüphanesi," GitHub deposu, 2014. [Çevrimiçi]. Erişim: <https://github.com/TsudaKageyu/minhook>
- [6] M. Sikorski ve A. Honig, Pratik Kötü Amaçlı Yazılım Analizi. San Francisco, CA: No Starch Press, 2012.
- [7] T. Garfinkel, "Tuzaklar ve tehlikeler: Sanal makine içsel incelemesiyle pratik sorunlar," in NDSS, San Diego, CA, 2003, ss. 1–16.
- [8] B. Stroustrup, C++ Programlama Dili, 4. baskı. Upper Saddle River, NJ: Addison-Wesley, 2013.
- [9] M. Sutter ve A. Alexandrescu, C++ Kodlama Standartları. Upper Saddle River, NJ: Addison-Wesley, 2004.
- [10] B. Stroustrup, "İstisna güvenliği: Kavramlar ve teknikler," İstisna İşlemede Gelişmeler, Berlin: Springer, 2001, ss. 60–76.
- [11] E. Cheng, "Windows'ta iş parçacığı güvenli satırıcı kancalama," in DEF CON 27, Las Vegas, NV, 2019.